CORE JAVA SERIES — DAY 4

Flow of Control, Loops & Jump Statements

How Java breaks out of plain line-by-line execution — Selection (if/switch), Iteration (while/do-while/for), and Jump statements (break/continue/labels), plus why switch can outperform if-else-if.

Flow of Control: The Big Picture

By default, code executes **sequentially** — statement 1, then statement 2, then statement 3, and so on. But certain keywords break this normal line-by-line flow and introduce an **alternate flow of control**. Java groups these into three categories:

Selection	Iteration	Jumps
if, if-else, switch	while, do-while, for (loops)	break, continue

The if Statement

An `if` statement gives your program a choice — like a fork in a road. Syntax: `if (expression) { ...do something... }`. The code block only executes if **expression evaluates to true** — the expression must always resolve to a `boolean` (true or false).



If false, the block is simply skipped and execution continues after it.

if-else

Adding `else` gives exactly two paths — the `if` block runs when the expression is true, the `else` block runs when it's false. Only one of the two ever executes.

> **Curly braces are optional** for a single statement inside if/else — but omitting them is not considered good practice, since adding a second statement later would silently break the logic if you forget to add braces back.

Nested if

An `if` can contain another `if` inside it (nesting), to any depth. Each inner `if` can have its own optional `else`. However, **good practice caps nesting at 1–2 levels** — deeper nesting makes code hard to read. Often, a nested condition can be flattened using a logical `&&` instead: `if (i > 5 && i < 10)` replaces two separate nested ifs checking the same range.

if-else-if Ladder

When there are more than two possible paths, chain conditions: `if...else if...else if...else`. Java checks each condition top to bottom and executes the **first one that's true** — then skips the entire rest of the ladder, jumping straight past it. A final plain `else` (optional) catches the case where none of the conditions matched.

> **Ladder vs. independent ifs:** In a true ladder, once one branch fires, none of the later `else if` / `else` branches are even checked. But separate, independent `if` statements (not chained with else) are each evaluated on their own — so more than one could fire for the same input. This distinction matters a lot in real code.

Active Recall — Part 1

Q1. What must an if statement's expression evaluate to?

A. A boolean value — either true or false.

Q2. Why is it good practice to always use curly braces with if, even for a single statement?

A. Because if a second statement is added later without remembering to add braces, only the first statement will actually be conditional — a common source of bugs.

Q3. In an if-else-if ladder, what happens once one condition is found true?

A. That block executes, and the rest of the ladder (all remaining else-if/else branches) is skipped entirely — control jumps past the whole ladder.

Q4. How is a nested if (i > 5) { if (i < 10) {...} } different from if (i > 5 && i < 10) {...}?

A. They achieve the same result, but the && version avoids unnecessary nesting and is considered better practice for readability.

The switch Statement

Similar in purpose to an if-else-if ladder, but with different syntax and (in the right circumstances) better performance. Basic structure:

```
switch (expression) { case value1: ...; break; case value2: ...; break; default: ...; break; }
```

Java compares the switch expression against each `case` value for equality; when a match is found, execution starts there and continues until a `break` is hit (or the switch block ends). `default` runs if no case matches.

Why break Matters — Fall-Through

Without break: if you omit `break` at the end of a case, execution "falls through" into the next case's code — regardless of whether that next case's value matches — continuing until a break is found or the switch ends. This can be intentional (executing several cases together) but is usually a bug if forgotten.

switch's Restrictions

- The switch expression must evaluate to `byte`, `short`, `int`, `char`, or an `enum` — never `boolean` (or any other type).
- Case values must all be **unique** — duplicate cases are a compile error.
- After JDK 7, `String` can also be used as a switch expression. After JDK 14, switch received major syntax enhancements (a separate topic).

switch vs. if-else-if Ladder

- **Test type:** switch can only test for **equality** against a value. An if-else-if ladder can test any boolean expression — equality, inequality, ranges, compound conditions, etc.
- **Efficiency:** switch can be more efficient because of how the compiler optimizes it internally (see below).

Why switch Can Be Faster: Jump Tables

An if-else-if ladder checks conditions **one at a time, in order** — worst case, that's O(n) comparisons before finding a match. A switch statement's compiler can instead build a **jump table** — conceptually like an array — that lets it jump **directly** to the matching case without testing each one sequentially.

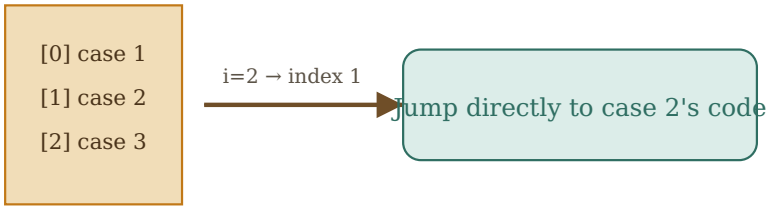


Table switch: with dense, close-together case values, the compiler builds an array-like table for O(1) direct access.

Table Switch vs. Lookup Switch

The compiler picks between two strategies based on how the case values are distributed:

- **Table Switch:** used when case values are **dense** (close together, e.g. 1, 2, 3). Builds a large direct-index array — O(1) jump to any case.
- **Lookup Switch:** used when case values are **sparse** (far apart, e.g. 1, 1000, 100000). Building a huge array would waste memory, so instead it stores just the actual case entries and locates the right one via something like **binary search** — O(log n) instead of O(1), but still better than a plain ladder's O(n).

> **Bottom line:** switch is not always faster in the same way — dense cases get near-instant O(1) jumps; sparse cases get O(log n) binary-search-like lookup. Either way, both beat the O(n) sequential checking of an if-else-if ladder.

Nested switch

Just like if statements, a `switch` can contain another `switch` inside one of its cases — nested to any depth, though (as always) deep nesting should be avoided for readability unless genuinely needed.

Active Recall — Part 2

Q1. What happens if you forget to write break at the end of a switch case?

A. Execution "falls through" into the next case's code regardless of whether it matches, continuing until a break or the end of the switch.

Q2. What data types can a switch expression evaluate to?

A. byte, short, int, char, enum, and (since JDK 7) String — never boolean.

Q3. What's the key testing-capability difference between switch and an if-else-if ladder?

A. switch can only test equality against fixed values; an if-else-if ladder can evaluate any boolean expression (ranges, inequalities, compound logic).

Q4. What determines whether the compiler builds a table switch or a lookup switch?

A. Whether the case values are dense (close together → table switch, O(1)) or sparse (far apart → lookup switch, O(log n) via binary-search-like lookup).

Iteration Statements (Loops)

A loop repeats a block of code a certain number of times (or until a condition changes), rather than executing it once. Java has three loop types:

while	do-while	for
Checks condition first, then runs body	Runs body first, then checks condition	Init, condition, and increment in one line — most commonly used

while Loop

> `while (expression) { ...do something... }` — the boolean expression is checked **before** each iteration; the body runs only as

long as it stays true.

Example: printing 1 to 10 — `int i = 1; while (i <= 10) { System.out.println(i); i++; }`. Forgetting to update `i` inside the loop creates an **infinite loop**, since the condition never becomes false.

do-while Loop

`do { ...do something... } while (expression);` — the body runs **at least once** before the condition is even checked, since the check happens at the end.

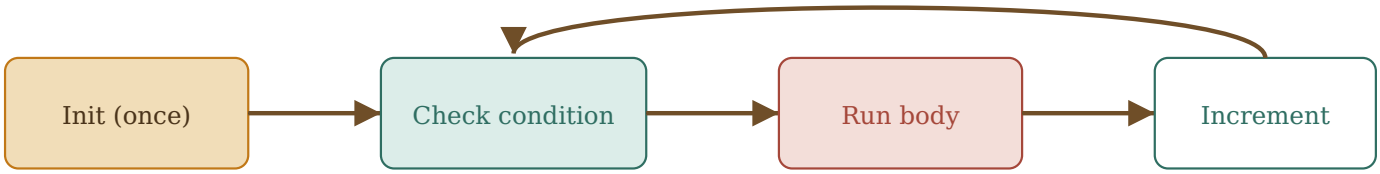
Key difference from while: if the condition is already false before the loop starts, a while loop never executes its body even once — but a do-while loop always executes its body at least once, regardless.

Best use case: menu selection. You want to show the user a menu (Play Game / Load Saved Game / Exit) at least once before checking what they chose — a do-while loop is the natural fit.

for Loop

Packs three parts into one line: `for (initialization; condition; increment/decrement) { ...do something... }`.

- **Initialization** — runs once, before the loop starts (e.g. `int i = 1`).
- **Condition** — checked before every iteration; loop continues while true.
- **Increment/decrement** — runs after each iteration's body finishes, before the next condition check.



for loop flow: initialize once → check → run body → increment → recheck, repeating until the condition fails.

A for loop can also run in **reverse** (start high, decrement), and any of its three parts can be a compound expression using **commas** — e.g. `for (int i = 1, j = 1; i <= 10; i++, j += 2)` lets you track two variables at once, each incrementing differently.

All three parts of a for loop are optional — `for (;;) { ... }` is perfectly valid syntax and creates an infinite loop, since nothing ever stops it.

Why Loop Variables Are Usually int

Even though a loop might only need to count to 10 (fitting easily in a `byte`), loop counters are almost always declared as `int` — never `byte` or `short`. Reason: due to type promotion, byte/short get promoted to int internally anyway during any arithmetic (like incrementing), so declaring them as int from the start avoids any wasted conversion and matches real-world convention.

Nested Loops

A loop can contain another loop inside it. For every single iteration of the **outer** loop, the **entire inner** loop runs to completion before the outer loop advances.

Time complexity: if the outer loop runs n times and the inner loop runs m times for each outer iteration, the total number of inner-body executions is **n × m**. When both loops run the same number of times (n), this becomes $O(n^2)$.

Pattern Printing

A classic use of nested loops: the outer loop controls how many **rows** to print; the inner loop controls how many **items per row** — often tied to the outer loop's current value (e.g. row 3 prints 3 stars). This is one of the best ways to build comfort with nested loop logic.

Jump Statements: break and continue

break

`break` immediately exits the loop (or switch) it's inside — no further iterations run, and control jumps to the first line right after the loop. Commonly used to exit a loop early once some condition is dynamically met (e.g. stopping a prime-check loop the moment a divisor is found, instead of needlessly checking the rest).

continue

`continue` skips the rest of the current iteration's body and jumps straight back to the loop's next iteration (re-checking the condition first) — it does not exit the loop, unlike break. Commonly used to skip specific values (e.g. skip even numbers, only print odd ones) without needing a full extra if-block wrapping the "do something" logic.

> **break vs continue in one line:** break says "stop the whole loop now." continue says "skip just this round, keep looping."

break/continue in Nested Loops

> **Default scope:** a plain `break` or `continue` inside a nested loop only affects the **innermost** loop it's written in — the outer loop keeps running normally.

Labels: Naming Loops

Any loop (or even a plain code block) can be given a name using a **label**: `outer: for (...) { inner: for (...) { ... } }`. Then `break outer;` or `continue outer;` can target the **outer** loop specifically from inside the inner one — instead of only ever affecting the innermost loop by default.

> **Example effect:** a labeled `break outer;` fired from inside the inner loop exits *both* loops entirely — useful when a single deeply-nested condition should stop the whole nested structure at once, not just the current inner pass.

Active Recall — Part 3

Q1. What's the key difference between a while loop and a do-while loop?

A. while checks the condition before running the body (may run zero times); do-while runs the body first and checks after (always runs at least once).

Q2. Why is do-while the best fit for a menu-selection screen?

A. Because you want to show the menu to the user at least once before any condition about their choice is even checked.

Q3. Are all three parts of a for loop (init, condition, increment) required?

A. No — all three are optional; for(;;) is valid Java and creates an infinite loop.

Q4. Why are loop counters almost always declared as int rather than byte or short?

A. Because byte/short get automatically promoted to int during arithmetic anyway (type promotion), so declaring as int avoids unnecessary conversion and matches convention.

Q5. If an outer loop runs n times and an inner loop runs m times per outer iteration, what's the total time complexity?

A. O(n × m) — or $O(n^2)$ when both loops run the same number of times.

Q6. What's the difference between break and continue?

A. break exits the loop entirely; continue skips only the rest of the current iteration and moves to the next one.

Q7. By default, which loop does break/continue affect in a nested loop, and how can you target the outer loop instead?

A. By default, only the innermost loop. Using a label (e.g. outer: for(...)) lets you write break outer; or continue outer; to target the outer loop specifically.
